

Guía de estudio de la clase de Aprendizaje Machín

Acoyani Garrido Sandoval

Septiembre de 2026

Índice general

1. Conceptos previos	2
1.1. Métricas MSE y R^2	2
1.2. Fitting, entrenamiento-prueba y leakage	3
1.2.1. Leakage	4
1.3. Limpieza de datos	5
1.4. Entrenamiento y Prueba	8
1.4.1. Hallazgos	9
1.5. Calidad de datos	9

Capítulo 1

Conceptos previos

1.1. Métricas MSE y R^2

El error cuadrático promedio es una de las métricas más básicas para medir la precisión de la predicción de un modelo de aprendizaje machín. Consiste en aplicar la fórmula:

$$MSE = \frac{1}{n} \sum_{i=1}^n [y_i - \hat{y}_i]^2$$

donde y_i es alguno de mis datos $y = \{y_1, y_2, \dots, y_n\}$ y $\hat{y}_i$ es alguno del conjunto $\hat{y}$ de datos predichos.

Es decir, es la sumatoria de la diferencia entre todos mis datos y el dato predicho, elevado al cuadrado para que se haga positivo y así la sumatoria no dé 0 en caso de que haya una distribución normal en el error de los datos. Es la medida más directa, pero padece de hacerse muy grande en caso de que mis datos sean números muy grandes por estar todo elevado al cuadrado.

Para remediar este problema, la segunda métrica básica consiste en una comparación divisiva del MSE de la predicción contra el MSE del modelo más sencillo de todos, que es el promedio. ¿Por qué el promedio? Porque, como lo vi en la clase de estadística multivariable, el promedio es la función básica que minimiza el error cuadrático acumulado (se obtiene de sacar la derivada del MSE con respecto a y e igualarla a 0). Entonces, si mi predicción reduce el MSE más de lo que el simple promedio es capaz, el R^2 va a ser más grande.

Entonces, con la y y $\hat{y}$ previamente definidas como mis datos de verdad y mis predicciones, defino ahora $\bar{y}$ como el promedio de mis datos y saco:

$$R^2 = 1 - \frac{\sum_{i=1}^n [y_i - \hat{y}_i]^2}{\sum_{i=1}^n [y_i - \bar{y}]^2}$$

Este número tiene 3 escenarios:

- La predicción fue perfecta, lo que causó que el MSE de mis datos de verdad contra los predichos sea 0, lo que hace que la división entera se haga 0, lo que hace que al 1 se le reste 0. **$R^2 = 1$ quiere decir una predicción perfecta.**
- La predicción fue tan mala como simplemente sacar el promedio. Eso causa que el MSE de la predicción sea igual que el MSE del promedio, lo que hace que la división se haga 1, lo cual se le resta al otro 1. **$R^2 = 0$ quiere decir que la predicción supuestamente no fue mejor que el promedio.**
- La predicción fue tan absurdamente mala que su MSE resultó ser mayor que el del promedio. El MSE de la predicción es el numerador de la división. Entonces, el numerador será más grande que el denominador, y eso hace que la división dé algo mayor que 1, lo cual se resta al otro 1, haciendo el resultado negativo. **$R^2 < 0$ quiere decir que la predicción fue peor que el promedio.**

Normalmente queremos que nuestro R^2 esté tan cerca de 1 como sea posible, pues en términos numéricos, $R^2 = 1$ quiere decir que nuestro modelo predijo todo. Sin embargo, como también ya sabemos y como se supo varias décadas después de la Comisión Stiglitz-Sen-Fitoussi, reducir algo tan complejo como la exactitud de nuestra predicción a una sola métrica es perdernos de muchos detalles.

1.2. Fitting, entrenamiento-prueba y leakage

El proceso de **fitting** (ajuste) o **entrenamiento** es el proceso de ajustar los parámetros internos de nuestro modelo de aprendizaje automático para que éste capture los patrones de nuestro conjunto de datos.

Los pasos básicos del proceso de fitting son:

1. **Definir una función objetivo:** Con frecuencia también conocida en la literatura como *función de costo* o *función de pérdida*, es una función que mide la discrepancia entre las salidas predichas y los datos verdaderos. El MSE es la función más sencilla y común de pérdida.
2. **Optimizar:** Con alguno de los miles de algoritmos que miles de científicos de datos han desarrollado para resolver todo tipo de problemas de esta ciencia multibillonaria, calculamos los parámetros internos óptimos que minimizan la función de pérdida. El proceso con frecuencia es iterativo,

pero no necesariamente: si nuestro modelo es regresión lineal, ésta tiene solución analítica.

3. **Evaluar:** Se prueba el modelo ajustado contra los **datos de validación**, con el fin de garantizar que éste generalice bien en vez de tan sólo memorizar ejemplos de entrenamiento.

De ahí se desprende que **los datos deben dividirse en conjunto de entrenamiento y conjunto de validación**. Esto se debe a que saber a priori el R^2 de un modelo ya ajustado no es suficiente para determinar si éste es capaz de generalizar; esto, ya que existe el escenario del **overfitting** (sobreajuste), que puede describirse como que el modelo se alineó demasiado fuertemente a nuestros datos, y sus parámetros internos terminaron reproduciendo más el ruido y las anomalías de nuestros datos que los datos en sí; es equivalente en machine learning a machetear un libro para un examen en vez de entender los conceptos, causando como resultado ser incapaz de contestar la pregunta si ésta se expresa de forma diferente.

Entonces, al dividir nuestros datos en entrenamiento y *training*, podemos identificar el *overfitting* y *underfitting* a priori, sin tener que esperar a nuestros datos reales; y esto lo hacemos a través del R^2 de nuestro modelo contra los datos de entrenamiento y contra los datos de prueba.

- **R^2 alto en Entrenamiento y Testing:** Este es el escenario ideal, y quiere decir que el modelo aprendió los patrones de Entrenamiento (alto R^2 ahí) y fue capaz de replicarlos en Testing (alto R^2 ahí). Un R^2 saludable a ojo de buen cubero es entre 0.8 y 0.9.
- **R^2 alto en Entrenamiento, bajo en Testing:** Overfitting; el modelo se ajustó por completo a los datos de Entrenamiento, pero no fue capaz de ajustarse a Testing. Hay muchas diversas causas: modelos que tienden a hacer overfitting, tienen demasiadas capas o parámetros, el dataset tiene ruido o outliers, o le hicimos demasiado entrenamiento...
- **R^2 bajo en Entrenamiento y Testing:** Underfitting; el modelo no puede ajustarse. Tal vez el modelo sea demasiado simple (la regresión lineal difícilmente ajusta en variables cuya relación no es lineal), tal vez nos haga falta más entrenamiento, tal vez haya outliers y ruido...

1.2.1. Leakage

El *leakage* ocurre cuando, por alguna razón, hemos usado datos ajenos al conjunto de Entrenamiento para entrenar nuestro modelo. El resultado que eso trae es que el modelo “hace trampa”, y trae como resultado que el R^2 en entrenamiento y testing parece estar fabulosamente alto, para que el modelo termine colapsando en Producción.

Una forma común de leakage es *contaminación cruzada entre Entrenamiento y Testing*, que es cuando Entrenamiento de alguna forma termina con datos

representativos de Testing. Por ejemplo, si relleno los nulos de mis datos de Entrenamiento con la mediana de todo mi dataset, **estoy contaminando mis datos de Entrenamiento con datos de Testing**, porque la mediana de todo mi dataset describe también los datos de Testing, y con eso el modelo tiene a priori una idea de cómo son los datos de Testing y se ajusta a eso, ya que los modelos de machine learning son para ajustarse; pero en el proceso, aprende a ajustarse únicamente a los datos que tengo, y se desconoce cuánto se ajustará a los datos que reciba ya en Producción.

Entonces, **el modelo no debe saber absolutamente nada de qué hay en Testing**. Se debe entrenar como si los datos de Testing no existieran todavía, como si Entrenamiento fueran todos sus datos y Testing fuera su puesta en servicio. En consecuencia, todas las transformaciones de datos (imputación, escalamiento, eliminación de outliers, feature engineering...) deben hacerse únicamente sobre Entrenamiento. *Corolario importante:* en línea con que los datos de Testing simulan la puesta en Producción del modelo, *no se deben tumbar los outliers de Testing si los tumbé en Entrenamiento*; pues Testing es una simulación de la realidad, y la realidad me va a tirar outliers.

1.3. Limpieza de datos

Trabajando sobre la práctica 2, comenzamos leyendo nuestro archivo y caracterizándolo un poco:

```
1 import pandas
2
3 df = pandas.read_csv("HW2_training.csv")
4 print(df.columns.tolist())
5 print()
6 print(df.dtypes)
7 print()
8 print(df.head(3))
9 print()
10 print(df.isna().sum())
```

El resultado es:

```
1 20.0
2 -3.0
3 [' X0 ', ' X1 ', ' X2 ', ' X3 ', ' X4 ', ' X5 ', ' X6 ', ' X7 ', ' X8
   ↪ ', ' Y ', 'Unnamed: 10']
4
5 X0      object
6 X1      object
7 X2      object
8 X3      object
9 X4      object
10 X5      object
```

```

11 X6          object
12 X7          object
13 X8          object
14 Y          object
15 Unnamed: 10 float64
16 dtype: object
17
18           X0           X1           X2  \
19 0  0.3745401190000  0.9507143060000  0.7319939420000
20 1  0.5986584840000  0.1560186400000  0.1559945200000
21 2  0.7080725780000  0.0205844940000  0.9699098520000
22
23           X3           X4  X5           X6
24 0  672,702.9942000000000  796,681.3972000000000 NaN  0.0000005310000
25 1  624,874.0996000000000  571,745.9831000000000 NaN  0.0000018100000
26 2   51,835.7991200000000  548,858.6651000000000 NaN  0.0000002230000
27
28           X7           X8           Y  Unnamed:
29 0  0.0000001720000  5.0000000000000  609,366.9835000000000
30 1  0.0000006130000  5.0000000000000  379,419.9478000000000
31 2  0.0000000729000  5.0000000000000  40,589.5357900000000
32
33 X0          5
34 X1          5
35 X2         11
36 X3          3
37 X4          2
38 X5         458
39 X6          5
40 X7          3
41 X8          2
42 Y         159
43 Unnamed: 10 808

```

De entrada podemos apreciar diversas cosas que dificultan trabajar con los datos de nuestro archivo: los nombres de las columnas están rodeados de espacios, hay una columna llamada *Unnamed: 10* que no parece tener nada, y vemos que muchas columnas están en formato numérico de Excel: entre los primeros datos de X3 podemos ver el número “ 483,039.6976000000000 ” rodeado de espacios y en notación de millares, y luego el número “(53.8607753100000)”, donde los paréntesis son notación de contabilidad americana para números negativos (no forma parte de las GAAP, pero sí lo requieren algunas organizaciones como la SEC).

Entonces, tenemos que limpiar nuestros números:

```
1 df.columns = df.columns.str.strip() # Quitamos espacios en columnas
2 df = df.drop(columns=["Unnamed: 10"]) # Tumbamos columna vacía
```

Para convertir nuestros números de string a número, necesitamos una función que convierta un string a un float:

```
1 def convertir_str_a_float(_param_str_numero):
2     # Bakayoke: ¿no es un string? de ser así, aborto
3     if type(_param_str_numero) is not str: return _param_str_numero
4     # Caso 1: espacio en blanco alrededor
5     _param_str_numero = _param_str_numero.strip()
6     # Caso 2: quitar comas
7     _param_str_numero = _param_str_numero.replace(",", "")
8     # Caso 3: ¿hay paréntesis? cambio por un -
9     if ( _param_str_numero.find('(') != -1 ):
10         _param_str_numero = _param_str_numero.replace('(', '-')
11         _param_str_numero = _param_str_numero.replace(')', '')
12     # Convierto a float; si todavía no puedo después de este proceso
13     # de conversión, me rindo y tiro None
14     try:
15         resultado = float(_param_str_numero)
16     except ValueError:
17         resultado = None
18     return resultado
```

Probamos la función con 4 valores.

```
1 print(convertir_str_a_float('609,366.9835'))
2 print(convertir_str_a_float('(158,355.4276)'))
3 print(convertir_str_a_float('0.3745401190'))
4 print(convertir_str_a_float(''))
```

Los resultados son:

```
1 609366.9835
2 -158355.4276
3 0.374540119
4 None
```

Y luego aplicamos esa función a cada elemento de nuestro Dataframe.

```
1 df = df.map(convertir_str_a_float)
```

Revisamos cómo quedó nuestro DF:

```
1 print(df.dtypes)
2 print()
3 print(df.isna().sum())
```

```
4 print()
5 print(df.shape)
```

El resultado es:

```
1 X0    float64
2 X1    float64
3 X2    float64
4 X3    float64
5 X4    float64
6 X5    float64
7 X6    float64
8 X7    float64
9 X8    float64
10 Y     float64
11 dtype: object
12
13 X0      5
14 X1      5
15 X2     11
16 X3      3
17 X4      2
18 X5    458
19 X6      5
20 X7      3
21 X8      2
22 Y     159
23 dtype: int64
24
25 (808, 10)
```

Y con esto nuestros datos ya están limpios: ya todas las columnas son de tipo float64, la cantidad de NaNs es la misma, la figura de nuestro dataframe es la misma, los datos están bien limpiados.

1.4. Entrenamiento y Prueba

Una vez limpiados los datos, sigue hacer la división entre Entrenamiento y Prueba. (La semilla 42 es mandato del enunciado de la práctica).

```
1 from sklearn.model_selection import train_test_split
2 X = df.drop("Y", axis=1)
3 Y = df["Y"]
4 # 20% Test, 80% Entrenamiento
5 X_entreno, X_prueba, Y_entreno, Y_prueba = train_test_split(X, Y,
6     ↪ test_size=0.2, random_state=42)
7 print(X_entreno.shape, X_prueba.shape)
```

A continuación me topo con pared: en el código propuesto de la práctica 2 se sugiere usar regresión lineal; sin embargo, sabemos muy bien que la regresión lineal requiere datos llenos en toda la tabla. Esto quiere decir que no podemos usar una línea base de preprocesamiento. Debido a que tenemos que evitar hacer procesamiento de más, vamos a probar cuántos datos caen si le doy `dropna()`, la bien conocida función de PANDAS que tumba filas con valores faltantes. En línea con el principio de no contaminar Entrenamiento con información sobre Prueba, esta vez esto lo tengo que hacer únicamente sobre Entrenamiento. Pero antes, tengo que revisar cuántas filas me tumbará el `dropna()`:

```
1 len(X_entreno)
2 len(X_entreno.dropna())
```

Si pruebo eso con el conjunto de datos abreviado a 100 renglones, me quedarán apenas 21 datitos sobre los cuales hacer regresión lineal. Así que hay que usar el conjunto completo de 800 renglones y fería. El resultado que me da usar el juego completo es que 646 renglones de Entrenamiento se reducen a 279, todavía pocos pero suficientes estrictamente hablando.

Así que, sin más ni menos, tumbo los NaN.

```
1 X_entreno = X_entreno.dropna()
```

1.4.1. Hallazgos

1. Las columnas tenían espacios en sus nombres, y varios números en formato contable americano (millares separados por comas y números negativos en paréntesis). Con la función mencionada previamente, se convirtieron a float.
2. Una columna llamada *Unnamed: 10* estaba totalmente vacía, se tumbó.
3. Hay 159 renglones contiguos al final con la Y vacía. Se tumbaron antes de hacer la división.
4. X5 falta en 70 % del bloque que sí tiene Y pero está presente en 98 % del bloque que no tiene Y. Sin embargo, como el promedio, la mediana y la desviación estándar de Y son muy similares tanto en las filas que sí tienen X5 como en las que no tienen X5, no se puede decir que hay una correlación entre no haber X5 y el valor de Y; por lo tanto, debería ser seguro imputar X5 (imputar una variable cuyos huecos no están relacionados con otra variable o con su propio valor generalmente es seguro porque no sesga los resultados).

1.5. Calidad de datos

La calidad de datos es una de las dimensiones más importantes del aprendizaje machín, por motivo de la ley *GIGO – garbage in garbage out*, si entra basura sale

basura. Debido a que la IA y el ML no son conscientes por más que Daniel Moskowitz intente decir lo contrario, los modelos de ML no son capaces de “saber” qué es ruido y qué es datos; todo está en los datos de entrenamiento, y cuando la calidad de los datos está comprometida, los modelos aprenden asociaciones incorrectas o pierden patrones.

Comenzamos obteniendo un informe numérico simple acerca de las medidas de tendencia central y de dispersión de cada columna.

```
1 df.describe().T # Nos conviene ver ese informe en forma transpuesta
```

El resultado es: